Dynamic Programming

Questions and Answers

Question 1 - Playing with Numbers

Problem Statement

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Example 1: Input: 6 Output: 6

Explanation: There are 6 ways to represent number 6 with 1 and 3: 1+1+1+1+1+1, 3+3, 1+1+1+3, 1+1+3+1, 1+3+1+1, 3+1+1+1.

Input Format: First Line contains the number n.

Output Format: Print: The number of possible ways 'n' can be represented using 1 and 3.

Example

Input	Output
6	6

Answer (C program)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    for(int i = 0; i <= n; i++)
        dp[i] = 0;

    dp[0] = 1;

    for(int i = 1; i <= n; i++)
    {
        dp[i] = dp[i - 1];

        if(i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

Output of the program

Input	Output
6	6

Dynamic Programming Approach

1. Let dp[i] = number of ways to write i as an ordered sum of 1s and 3s.
2. Base case: dp[0] = 1 (one way to make 0: use nothing).
3. The last number used is either 1 or 3, so dp[i] = dp[i - 1] + dp[i - 3] (the second term only when i >= 3).
4. The answer is dp[n].

Time Complexity

Time complexity: O(n)
One loop from 1 to n with constant work per step. Space is O(n) for the dp array. (The plain recursion would be exponential.)

Question 2 - Playing with Chessboard

Problem Statement

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example: Input: 3 / 1 2 4 / 2 3 4 / 8 7 1 Output: 19

Explanation: Totally there will be 6 paths among that the optimal is. Optimal path value: 1+2+8+7+1 = 19.

Input Format: First Line contains the integer n. The next n lines contain the n*n chessboard values.

Output Format: Print Maximum monetary value of the path

Example

Input	Output
3 1 2 4 2 3 4 8 7 1	19

Answer (C program)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n][n];
    int dp[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    dp[0][0] = a[0][0];

    for (int j = 1; j < n; j++)
        dp[0][j] = dp[0][j - 1] + a[0][j];

    for (int i = 1; i < n; i++)
        dp[i][0] = dp[i - 1][0] + a[i][0];

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = a[i][j] + (dp[i - 1][j] > dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1]);
        }
    }

    printf("%d", dp[n - 1][n - 1]);
```

```
    return 0;
}
```

Output of the program

Input	Output
3 1 2 4 2 3 4 8 7 1	19

Dynamic Programming Approach

1. Let dp[i][j] = maximum money collected on a path from (0,0) to cell (i,j).

2. First row: dp[0][j] = dp[0][j-1] + a[0][j] (only moves right). First column: dp[i][0] = dp[i-1][0] + a[i][0] (only moves down).

3. Every other cell is reached from above or from the left: dp[i][j] = a[i][j] + max(dp[i-1][j], dp[i][j-1]).

4. The answer is dp[n-1][n-1].

Time Complexity

Time complexity: $O(n^2)$

Every cell of the n x n board is filled once with constant work. Space is $O(n^2)$ for the dp table.

Question 3 - Longest Common Subsequence

Problem Statement

Given two strings find the length of the common longest subsequence (need not be contiguous) between the two. Solving it using Dynamic Programming.

Example: s1: ggtabe s2: tgatasb The length is 4 (the common subsequence is g, t, a, b).

For example: Input: aab / azb Result: 2

Example

Input	Output
aab azb	2

Answer (C program)

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[1000], s2[1000];

    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n + 1][m + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            }
            else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                dp[i][j] = dp[i - 1][j] > dp[i][j - 1]
                           ? dp[i - 1][j]
                           : dp[i][j - 1];
            }
        }
    }

    printf("%d", dp[n][m]);

    return 0;
}
```

Output of the program

Input	Output
aab azb	2

Dynamic Programming Approach

1. Let dp[i][j] = length of the LCS of the first i characters of s1 and the first j characters of s2.
2. If i == 0 or j == 0, dp[i][j] = 0 (an empty string has no common subsequence).
3. If s1[i-1] == s2[j-1], the characters match: dp[i][j] = dp[i-1][j-1] + 1.
4. Otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]).
5. The answer is dp[n][m].

Time Complexity

Time complexity: O(n * m)
n and m are the lengths of the two strings. The table has (n + 1) x (m + 1) cells, each filled in constant time. Space is also O(n * m).

Question 4 - Longest Non-decreasing Subsequence

Problem Statement

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg: Input: 9 Sequence: [-1,3,4,5,2,2,2,2,3]

Explanation: the subsequence is [-1,2,2,2,2,3]

Output: 6

Example

Input	Output
9 -1 3 4 5 2 2 2 2 3	6

Answer (C program)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n], dp[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        dp[i] = 1;
    }

    int ans = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] <= a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
            }
        }

        if (dp[i] > ans)
            ans = dp[i];
    }

    printf("%d", ans);

    return 0;
}
```

Output of the program

Input	Output
9 -1 3 4 5 2 2 2 2 3	6

Dynamic Programming Approach

1. Let dp[i] = length of the longest non-decreasing subsequence that ends at index i. Start with dp[i] = 1 (the element alone).
2. For every earlier index j < i with a[j] <= a[i], we can extend that subsequence: dp[i] = max(dp[i], dp[j] + 1).
3. The answer is the largest value in dp[] (kept in ans).

Time Complexity

Time complexity: $O(n^2)$
Two nested loops (i and j < i). Space is O(n) for dp[]. A faster O(n log n) solution exists using binary search, but this DP version is the standard one.